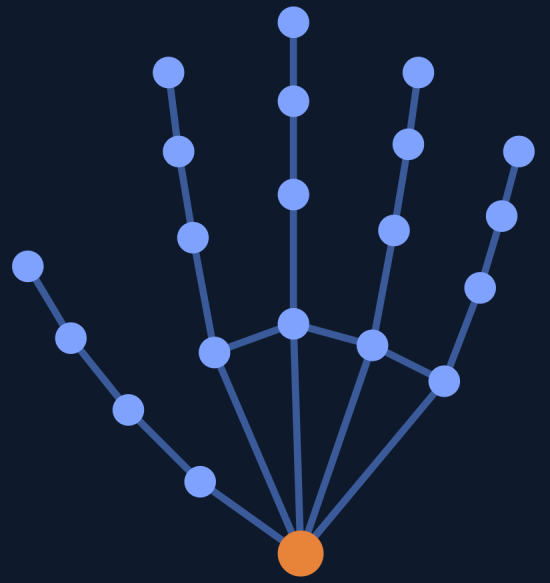
DEEP LEARNING LAB · FINAL PROJECT

SignSync

Continuous ASL-to-English translation with a part-aware graph neural network



Kashish Chelwani · PRN 24070126082

Symbiosis Institute of Technology, Pune · 27 October 2026

THE PROBLEM

Sign language carries meaning in many places at once

Hands, body and face

Handshape, movement, posture and facial expression all change the meaning.

No word boundaries

Continuous signing is one stream. The model must find where each sign starts and ends.

Not English grammar

ASL word order differs from English, so output cannot be a word-by-word mapping.

Approach: read poses, not pixels.

A skeleton is a graph of joints and bones, so a graph neural network is a natural fit.

DATASET

How2Sign: continuous ASL paired with English sentences

80+
hours of instructional ASL video

2,357
clips in the official test split

2,349
clips evaluated; 8 had no pose file

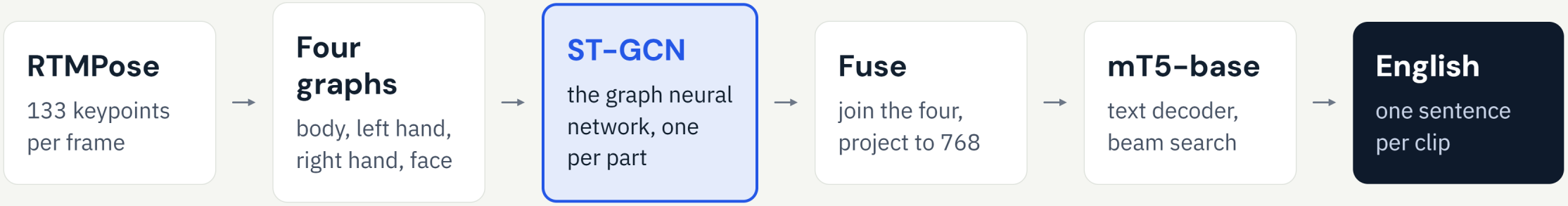
Input: 133 whole-body keypoints per frame, extracted by RTMPose.

Output: one English sentence per clip, scored against the reference.

Source: Duarte et al., How2Sign, CVPR 2021.

ARCHITECTURE

Poses in, English out: the whole pipeline



The decoder receives the prompt *“Translate sign language video to English:”* followed by the pose features, and four-beam search writes the sentence.

Blue marks the graph neural network: only 4.6 M of the model’s 588 M parameters.

THE GRAPH NEURAL NETWORK

Four skeleton graphs, one ST-GCN for each

9
Body joints

21
Left hand joints

21
Right hand joints

18
Face joints

- Spatial convolution mixes each joint with its neighbours along the bones
- Temporal convolution mixes each joint across frames
- The two hands share weights; body features feed the hand and face branches
- Mean over joints, join 4 × 256 = 1,024, project to 768 per frame

4.6 M
graph encoders

582 M
mT5-base

0.8 M
other layers

588 M
total parameters

REFERENCE DESIGN · THE WHOLE PROJECT

From video to a working translator, in eight stages

1
Collect
How2Sign videos paired with English sentences

2
Extract poses
RTMPose gives 133 keypoints per frame, saved as .npy

3
Prepare data
Manifest, train / val / test, four part graphs, at most 256 frames

4
Build model
Four ST-GCNs feeding mT5-base

5
Train
AdamW, cosine schedule, cross-entropy on the sentence

6
Tune
Small hyperparameter search, then decoding-only tuning

7
Evaluate
BLEU-4, ROUGE-L, bootstrap interval, control tests

8
Deploy
Web app, notebook, GitHub, hosted page

REFERENCE DESIGN · TRAINING

How the model is trained

Stage A · Pre-train

Pose-only, on large sign-video collections. Gives the graph encoder general sign knowledge.

Stage B · Fine-tune

On How2Sign for sign-to-English translation.

The mT5-base decoder starts from pre-trained weights from the four graph encoders (about 4.6 M parameters).

Optimizer	**AdamW (β 0.9, 0.999)**
Learning rate	**3e-4, warm-up then cosine decay**
Weight decay	**1e-4**
Epochs (fine-tune)	**20**
Batch size	**8, with gradient accumulation**
Input length	**up to 256 frames**
Loss	**cross-entropy on the English tokens**
Precision	**bf16 with gradient clipping 1.0**

REFERENCE DESIGN · TUNING

Two kinds of tuning

Hyperparameter search

Changes the model

- Random search: 6 trials × 3 epochs on about 500 clips
- Learning rate 1e-4, 3e-4, 1e-3
- Weight decay 0, 1e-4, 1e-2
- Dropout 0, 0.1, 0.3 · batch size 4 or 8
- Keep the best validation BLEU-4, then retrain on all data

Decoding-only tuning

Changes how text is generated; no retraining

- Beams: 1, 4, 8
- Repetition penalty: 1.0, 1.2, 1.5
- No-repeat n-gram size: off or 3
- Score validation BLEU-4 and the share of looping outputs
- Pick the best; report on the test split once

RESULTS · 2,349 TEST CLIPS

13.75 BLEU-4 on 2,349 test clips

13.75

BLEU-4

95% interval 13.04 to 14.47

30.7

ROUGE-L

my own implementation

14.5

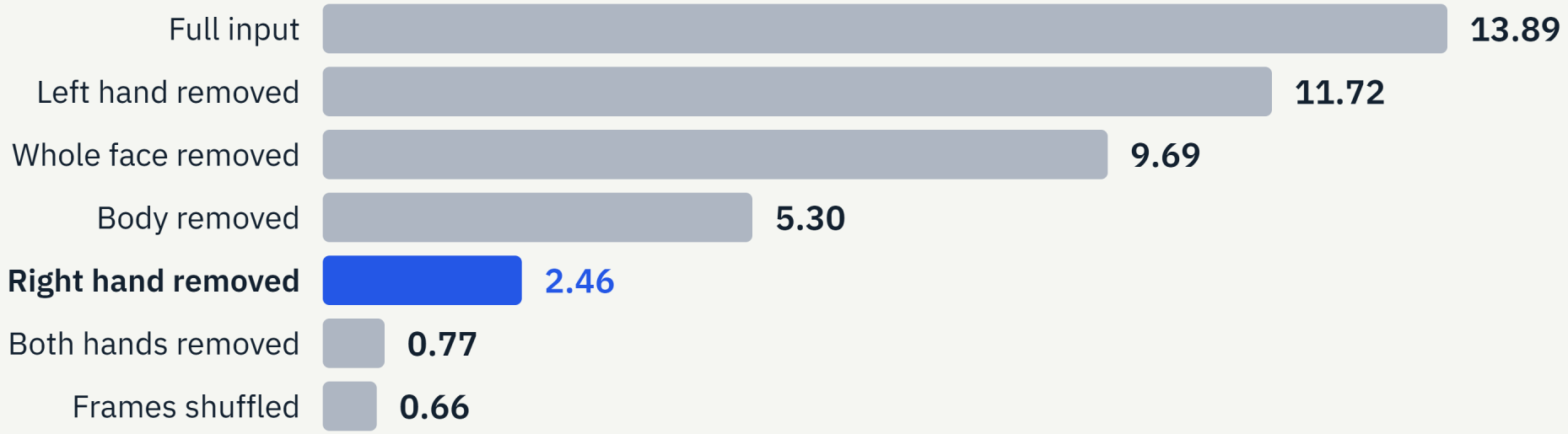
Published BLEU-4

Uni-Sign paper, ROUGE 34.3

Evaluation of the released pose-only model on the official How2Sign test split. The published BLEU-4 for the same setup is 14.5.

CONTROL TESTS · 500 CLIPS · BLEU-4

Removing the right hand cuts BLEU-4 from 13.89 to 2.46



Almost nothing survives without the hands or the frame order, so the score comes from the pose, not from the language model alone.

Trained model, parts zeroed at test time, no retraining. Bars start at zero.

DEMONSTRATION

The walkthrough, in four steps

1

Code and notebook

Every stage traced, with outputs

2

GitHub

App, scripts, results, Colab notebook

3

Deployed site

Saved results and a recorded demo

4

Live run

Upload a clip in the app and translate

github.com/Kashish811/SignSync-asl-translation

huggingface.co/spaces/Kashish811/SignSync-asl-translation

The static site shows saved results; live translation runs in the app on Colab. A recorded demo is the backup.

REFERENCES

Sources used

- Li et al. (2025). Uni-Sign: Toward Unified Sign Language Understanding at Scale. arXiv:2501.15187
- Duarte et al. (2021). How2Sign: A Large-Scale Multimodal Dataset for Continuous ASL. CVPR, 2735–2744
- Jiang et al. (2023). RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose. arXiv:2303.07399
- Yan, Xiong and Lin (2018). Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition. AAAI
- Xue et al. (2021). mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. NAACL, 483–498
- Rust et al. (2024). Towards Privacy-Aware Sign Language Translation at Scale. arXiv:2402.09611
- Camgoz et al. (2018). Neural Sign Language Translation. CVPR, 7784–7793
- Papineni et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL, 311–318
- Post (2018). A Call for Clarity in Reporting BLEU Scores. WMT, 186–191
- Lin (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out, 74–81

QUESTIONS

Thank you

Kashish Chelwani · PRN 24070126082